

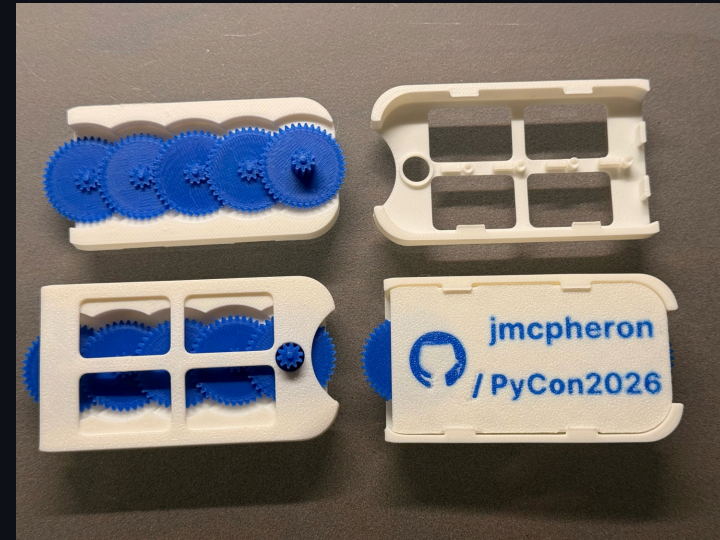
# **I Made a Gearbox Business Card, Then Made Python Explain It**

**Jason McPheron** · PyCon US 2026 · Lightning Talk

# Hi, I'm Jason.

---

- IT work, California community college — mostly student systems
- This talk is about what I do when I should be relaxing: **3D printing little mechanical things and then overthinking them with Python**
- First PyCon — wanted something physical to bring
- Something more interesting than "here's my GitHub"



# So I made a 3D printed business card.

---

It has my GitHub project on it.

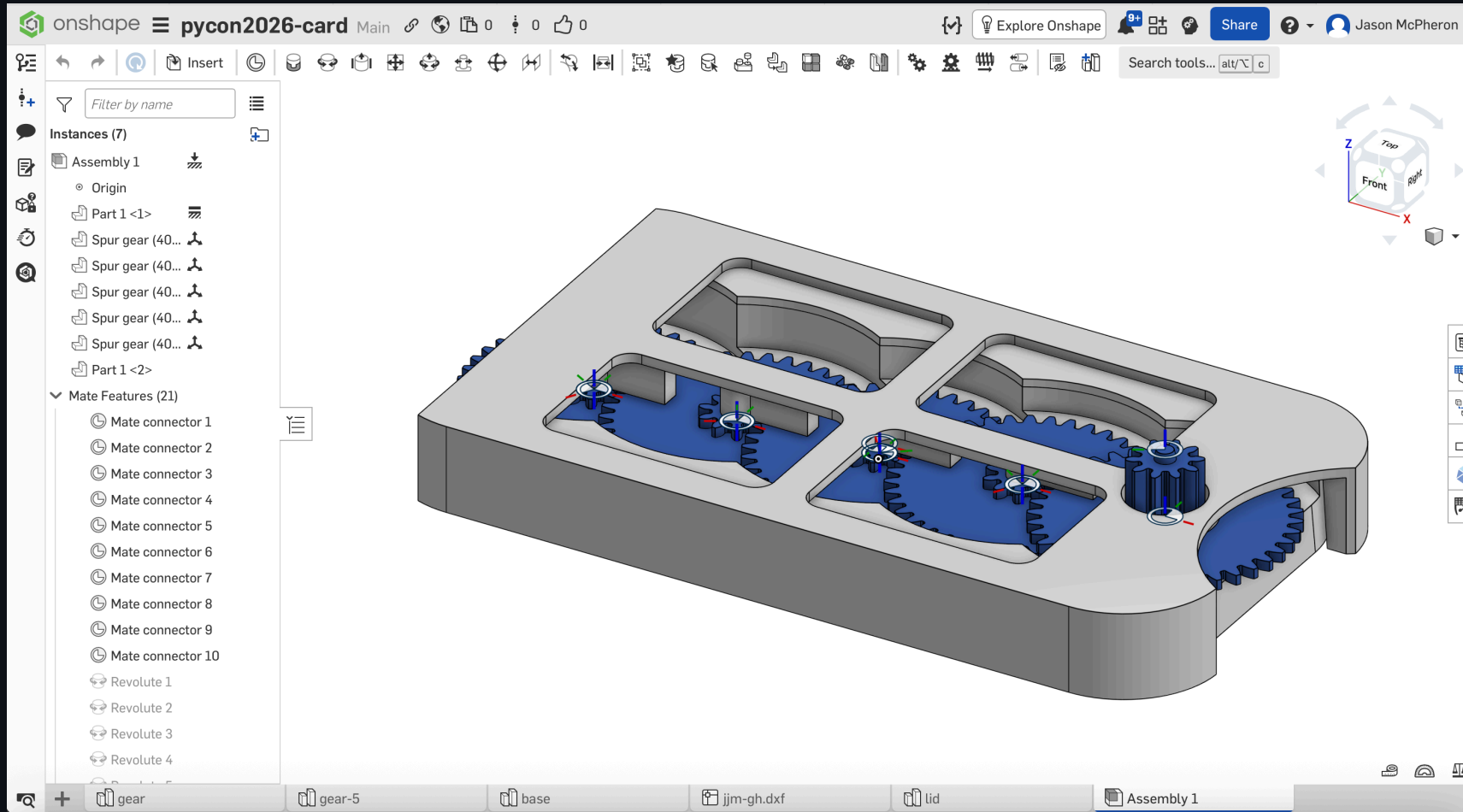
It also has a **tiny gear mechanism** built into it.

You can spin one gear with your thumb, and the motion moves through the card.

*Part business card. Part fidget toy. Part tiny mechanical demo.*

At first, that was the whole project. Make a cool little object. Print it. Bring it to PyCon.

# This is where I was happy with the design.



# The moment the project changed.

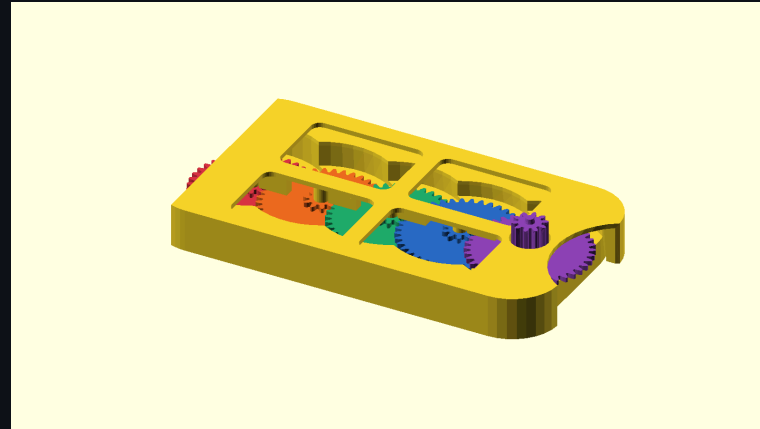
---

Once I had the card assembled, I exported it as a **STEP file**.

```
Onshape assembly → STEP export →  
GitHub repo
```

Now I had a **single file** that represented the assembled physical object.

I could put it in a repo. Version it. Run tools against it. Treat the physical object a little more **like a software project**.



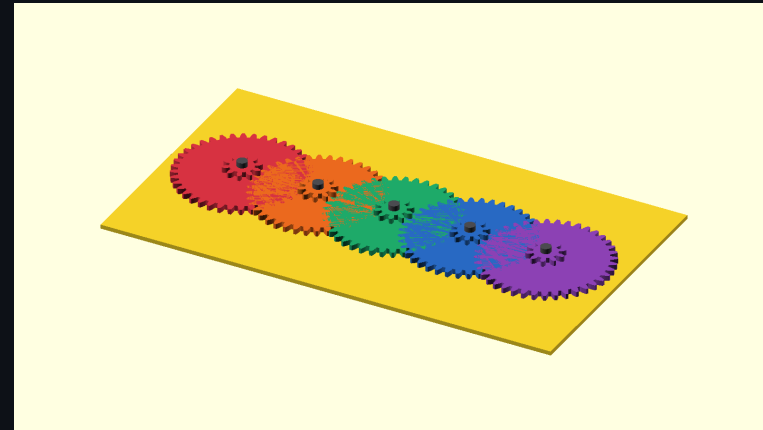
# Python around the STEP file.

---

Not replacing CAD.

## Tooling around the CAD export.

- Inspect the file
- Take the assembly apart
- Generate renders
- Make exploded views
- Animate the mechanism
- Keep notes, journals, tolerance experiments



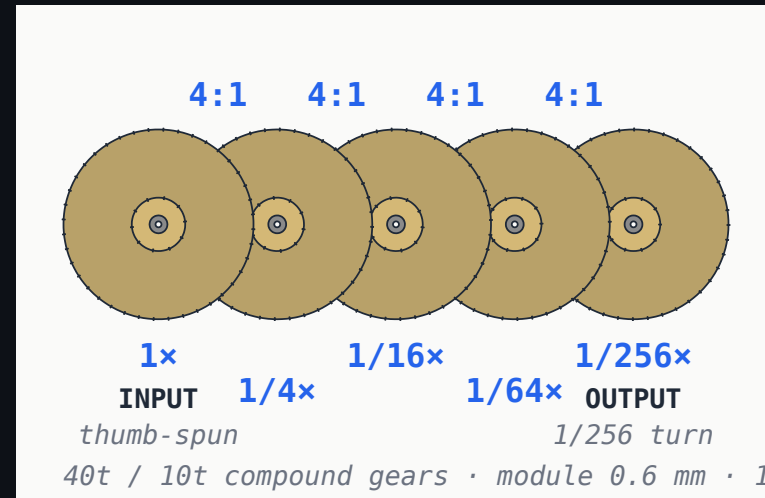
Yes, the gears clip through each other.  
If that bothers you — *stay tuned*.

# A STEP file is not an explanation.

GIFs show *what* the object does. They don't say *why* it's shaped that way.

So the repo also writes **explainer pages** — short prose with diagrams:

- `gear-ratios.md` — why  $32 \times 8$ , four times, gives 256:1
- `decoding-gears.md` — recovering tooth counts from the STL geometry itself
- `stacking.md` — why the gears sit in a staircase, not a stack
- `printing.md` — what changed when the printer, rudely, obeyed physics



# GitHub Actions for physical objects.

---

The cool part isn't that Python *can* make these. It's that I never run it.

Push a change to the STEP file or the gear constants. A workflow regenerates the GIFs, the explainer pages, the diagrams — and **commits them back to main**.

```
jmcpheron-card.step changes → build-card.yml → new exploded GIF, spin GIF, parts
```

```
src/explainers/card.py changes → build-explainers.yml → new pages + SVGs
```

*A README can quietly start lying about its object.*

*An auto-commit loop makes that a lot harder.*

**Remember the clipping gears?** Fork the repo, fix the spin renderer so each gear turns at its real ratio, push, and the workflow will rebuild the GIF for you. **PR welcome.**

# Paws for animals.

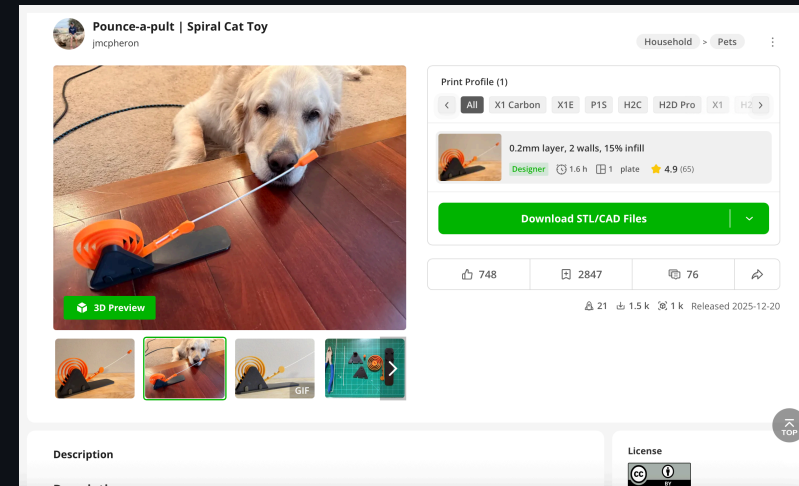
Before this card, I made a 3D printed **cat toy** for my mom.

That project is where I really started enjoying designing separate parts and assembling them.

The only problem:

**I do not have a cat.**

I have a golden retriever named **Bowie**.



The cat toy on MakerWorld. Bowie remains unimpressed.

# The tiny plastic maker version.

---

I am sure serious hardware fields have much more mature ways to do this.

Probably industry-standard workflows, best practices, blind spots I have not even learned enough to know I have.

Mine is:

- A 3D printed gearbox business card
- A public Onshape model
- A STEP export
- A Python CLI
- GitHub Actions making pictures

*Small enough to be playful.*

[github.com/imcpheron/pycon2026](https://github.com/imcpheron/pycon2026)

*Silly enough to experiment with.*

# Physical open source borrows from software.

---

Version control · Generated documentation · Repeatable builds

Reviewable changes · Public artifacts

And maybe: **little sanity checks that help the object explain itself.**

One more thing: I'm also curious about **licensing**.

For code, we have familiar open-source licenses.

For functional 3D design files — STEP files, things you can print and use — I'm less sure what the right answer is.

Does Creative Commons make sense? Is CC BY-SA the right model? What does *sharing* mean when someone can download the geometry, modify it, print it, remix it?

Making those questions **part of the project** instead of something I figure out later in private.

# Come find me.

---

Onshape · OpenSCAD · Python · gears · GitHub Actions

STEP files · licensing · cat toys · golden retrievers

*how much tooling is too much tooling for a tiny plastic business card*



**[github.com/jmcpheeron/pycon2026](https://github.com/jmcpheeron/pycon2026)** — I have cards, ask for one.

[github.com/jmcpheeron/pycon2026](https://github.com/jmcpheeron/pycon2026)